

Comparing Classical and Neural Models for Tweet Sentiment Analysis

Juan Miguel Pinos
Roma Tre University
Big Data
July 2026

Abstract

This project compares four approaches to three-class sentiment analysis on English tweets: TF-IDF with a linear support vector classifier, TextCNN, a bidirectional GRU with attention, and BERTweet. The models were trained and evaluated on the official TweetEval sentiment splits. Macro-F1 was used as the main metric because the class distribution is uneven, especially in the test set. TextCNN, BiGRU, and BERTweet were trained with three random seeds, and their final predictions were obtained by averaging the class probabilities of the three runs. BERTweet achieved the best test macro-F1, 0.718, followed by LinearSVC at 0.608, BiGRU at 0.600, and TextCNN at 0.589. The statistical analysis confirms that BERTweet is clearly better than the other systems, while the difference between LinearSVC and BiGRU is not significant. The results show that domain-specific pretraining is very useful for tweet classification, but the classical model remains a strong option when speed, size, and simplicity are important.

1 Introduction

Sentiment analysis assigns an opinion label to a piece of text. In this project, every tweet is classified as negative, neutral, or positive. Although the task is simple to describe, tweets are difficult to process because they are short, informal, and often contain mentions, hashtags, abbreviations, spelling variations, or indirect opinions. The neutral class is also difficult because it can contain words that appear in positive or negative messages without expressing a clear sentiment.

The aim of the project is to compare models with different levels of complexity. The first is a classical linear classifier based on TF-IDF features. The second is a convolutional neural network that learns local text patterns. The third is a recurrent model that processes the sequence in both directions and uses attention. The last is BERTweet, a transformer that was pretrained on a large collection of tweets and then fine-tuned for this task.

The comparison is not limited to the final score. The project also studies stability across random seeds, performance by class, results on different types of tweets, training and inference cost, and statistical significance. This makes it possible to discuss not only which model performs best, but also which model is more practical under different constraints.

2 Dataset

The experiments use the sentiment subset of TweetEval. It contains English tweets labeled as negative, neutral, or positive and provides fixed training, validation, and test partitions. These official splits were kept unchanged for all models.

Table 1: Dataset splits and class distribution.

Split	Rows	Negative	Neutral	Positive
Train	45,615	7,093 (15.6%)	20,673 (45.3%)	17,849 (39.1%)
Validation	2,000	312 (15.6%)	869 (43.5%)	819 (41.0%)
Test	12,284	3,972 (32.3%)	5,937 (48.3%)	2,375 (19.3%)

The training and validation sets have similar distributions. The test set is different: it contains a much larger proportion of negative tweets and fewer positive tweets. This creates a distribution shift between model development and final evaluation. The average text length also changes from 19.2 words in training to 14.9 in test. In addition, 809 test tweets contain an emoji, while none were detected in the training and validation sets.

The data quality checks found 29 duplicate texts inside the training set, no duplicates in validation or test, no missing texts, and no overlap between the three partitions. The duplicates were not removed because they are part of the official split and their number is very small compared with the full training set.

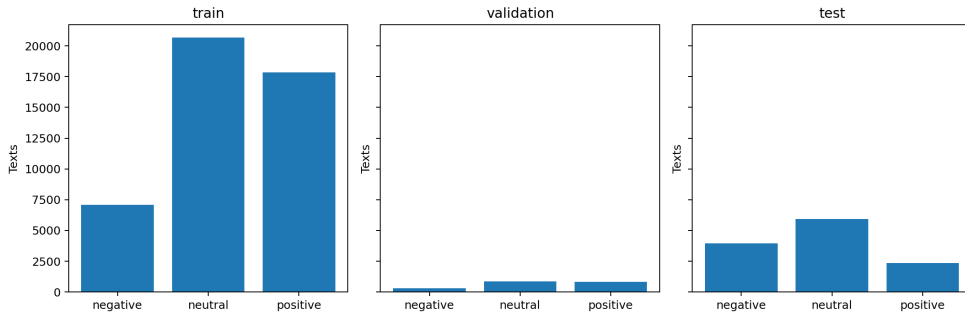


Figure 1: Class distribution in the official train, validation, and test splits.

3 Methodology

3.1 Preprocessing

Two preprocessing branches were used. For LinearSVC, TextCNN, and BiGRU, URLs were replaced with the token `url`, mentions with `user`, emojis were converted to textual descriptions, and hashtags were converted to tokens beginning with `hashtag_`. The text was lowercased, the contraction `n't` was expanded to `not`, and most symbols were removed while keeping exclamation and question marks.

BERTweet received lighter normalization. URLs were replaced with `HTTPURL` and mentions with `@USER`, which matches the format used by its tokenizer. The remaining text was kept close to its original form. This avoids changing the type of text on which the pretrained model learned its representations.

For TextCNN and BiGRU, a vocabulary was built only from the training set. Words appearing fewer than two times were excluded, and sequences were padded or truncated to 31 tokens, selected from the 99th percentile of the training lengths. BERTweet used its own tokenizer and a maximum length of 64 subword tokens.

3.2 TF-IDF and LinearSVC

The classical system combines word and character TF-IDF features. Word features use unigrams and bigrams, while character features use n-grams of length 3 to 5. Up to 90,000 word features

and 90,000 character features were retained. Character features are useful for spelling variations and word fragments that are common in social-media text.

Six LinearSVC settings were compared on the validation split. The values $C \in \{0.5, 1.0, 2.0\}$ were tested with and without balanced class weights. The best validation macro-F1 was obtained with $C = 0.5$ and balanced weights. The vectorizers and classifier were fitted only on the training set.

3.3 TextCNN

TextCNN learns 128-dimensional word embeddings from scratch. It applies three one-dimensional convolution layers with kernel sizes 2, 3, and 4. Each layer has 100 filters. Global max pooling selects the strongest feature from each filter, and the resulting 300 values are passed through dropout and a final linear layer.

The model was trained with AdamW, a learning rate of 0.001, balanced class weights, gradient clipping, and early stopping based on validation macro-F1. The maximum number of epochs was 15, although training normally stopped earlier.

3.4 BiGRU with Attention

The recurrent model uses 200-dimensional embeddings and a bidirectional GRU with 128 hidden units in each direction. Padding positions are excluded through packed sequences and an attention mask. The attention layer computes a weighted combination of all valid hidden states before the final classification layer.

The model was trained with AdamW, a learning rate of 0.0005, class weights, label smoothing of 0.05, gradient clipping, and early stopping. Compared with TextCNN, this model can combine information from different positions in the sentence, but it is also more sensitive to initialization and training order.

3.5 BERTweet

BERTweet is a transformer pretrained on English tweets. A new three-class classification head was added, and the complete model was fine-tuned on the TweetEval training split. Training used AdamW with a learning rate of 2×10^{-5} , weight decay of 0.01, a linear learning-rate schedule with 10% warm-up, label smoothing of 0.05, gradient accumulation, mixed precision, gradient clipping, and early stopping.

Unlike TextCNN and BiGRU, BERTweet does not learn language representations only from the labeled training set. It starts with knowledge obtained from a much larger collection of tweets. This is the main difference between the transformer and the two neural models trained from scratch.

4 Experimental Protocol

All experiments were executed in Google Colab with an NVIDIA T4 GPU. LinearSVC is effectively deterministic under the selected configuration. TextCNN, BiGRU, and BERTweet were each trained with random seeds 13, 42, and 77. For each neural architecture, the final ensemble averages the three predicted probability vectors and selects the class with the highest average probability.

The validation split was used to select the LinearSVC configuration and the best checkpoint of every neural run. The test set was then used for the final comparison. The main metric is macro-F1, which gives equal importance to the three classes. Accuracy, macro precision, macro recall, weighted F1, and per-class F1 were also calculated.

Uncertainty was estimated with 500 bootstrap resamples of the test set. Pairwise model differences in accuracy were evaluated with McNemar’s exact test. Six pairwise comparisons were performed, so the resulting p-values were adjusted with the Bonferroni correction.

Table 2: Main training settings.

Setting	Value
Environment	Google Colab, NVIDIA T4 GPU
Random seeds	13, 42, and 77
LinearSVC selection	Validation macro-F1 over six settings
TextCNN optimizer	AdamW, learning rate 0.001
BiGRU optimizer	AdamW, learning rate 0.0005
BERTweet optimizer	AdamW, learning rate 2×10^{-5}
Neural checkpoint selection	Best validation macro-F1 with early stopping
Final neural prediction	Average of probabilities from three seeds
Primary test metric	Macro-F1
Statistical analysis	500 bootstrap samples and exact McNemar tests

5 Results

5.1 Overall Performance

Table 3 shows the main results. BERTweet is clearly the best system, reaching 0.718 macro-F1. LinearSVC is second with 0.608, followed by BiGRU at 0.600 and TextCNN at 0.589. The two models trained from scratch do not improve over the classical baseline on the test set.

Table 3: Validation and test results.

Model	Val. F1	Accuracy	Macro P	Macro R	Macro F1
TF-IDF + LinearSVC	0.668	0.612	0.602	0.617	0.608
TextCNN ensemble	0.657	0.599	0.588	0.596	0.589
BiGRU ensemble	0.672	0.608	0.598	0.602	0.600
BERTweet ensemble	0.745	0.718	0.710	0.730	0.718

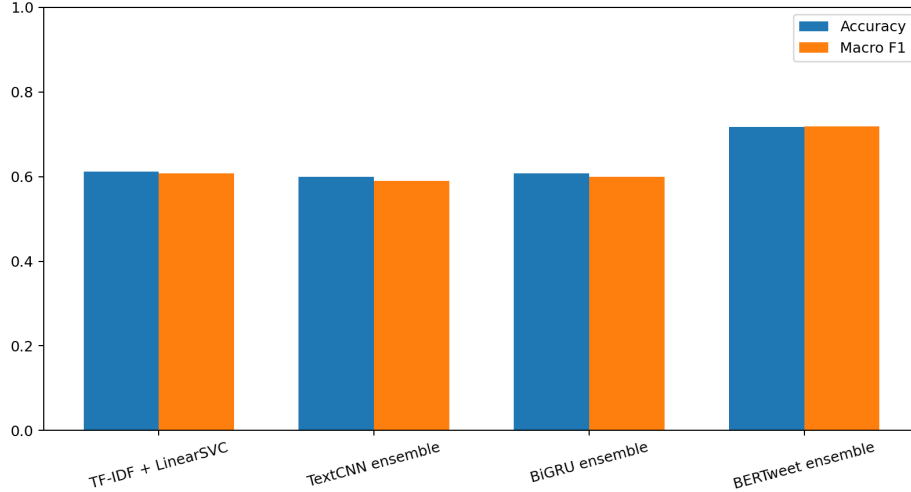


Figure 2: Test accuracy and macro-F1 for the four final systems.

BERTweet also has the best F1 for every class. The negative class has its highest score, 0.744. Its lowest class score is neutral at 0.703, but this is still clearly above the other systems. The hardest class is not the same for every model: negative is the weakest class for TextCNN and BiGRU, positive is slightly weaker for LinearSVC, and neutral is the weakest for BERTweet.

Table 4: Per-class F1 on the test set.

Model	Negative	Neutral	Positive
TF-IDF + LinearSVC	0.599	0.628	0.597
TextCNN ensemble	0.565	0.632	0.571
BiGRU ensemble	0.575	0.639	0.585
BERTweet ensemble	0.744	0.703	0.708

5.2 Stability Across Random Seeds

The individual BERTweet runs obtained a mean macro-F1 of 0.7137 with a standard deviation of only 0.0014. This is much more stable than BiGRU, whose standard deviation was 0.0105. TextCNN obtained a mean of 0.5640 with a standard deviation of 0.0078.

The ensembles improved all three neural systems. TextCNN increased from a mean individual score of 0.5640 to 0.5894, BiGRU from 0.5783 to 0.5996, and BERTweet from 0.7137 to 0.7184. The improvement is largest for TextCNN and BiGRU because their runs make more varied errors. BERTweet is already stable, so its ensemble gain is smaller.

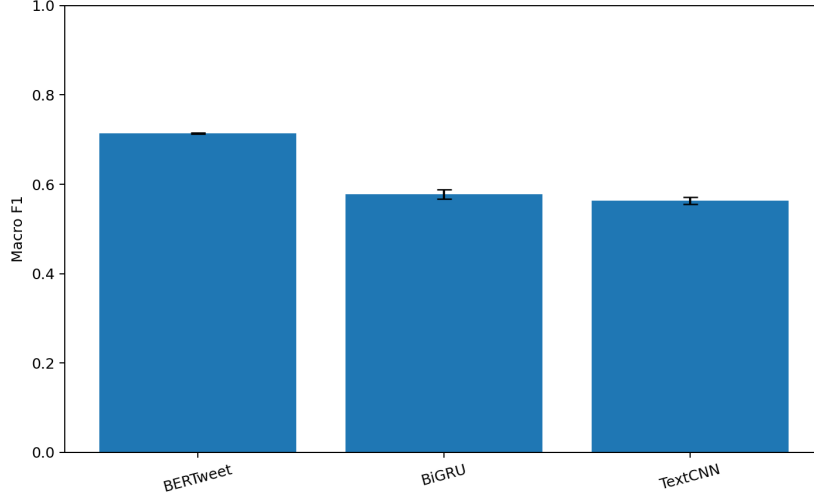


Figure 3: Mean test macro-F1 of individual neural runs with one standard deviation.

5.3 Training Behaviour

Figure 4 shows the validation macro-F1 for the representative seed 42. TextCNN improves quickly and reaches its best validation result after a small number of epochs. BiGRU takes longer and shows more variation. BERTweet starts from a much higher score and reaches its best result within four epochs. Early stopping ensures that the final prediction uses the best validation checkpoint instead of the final epoch.

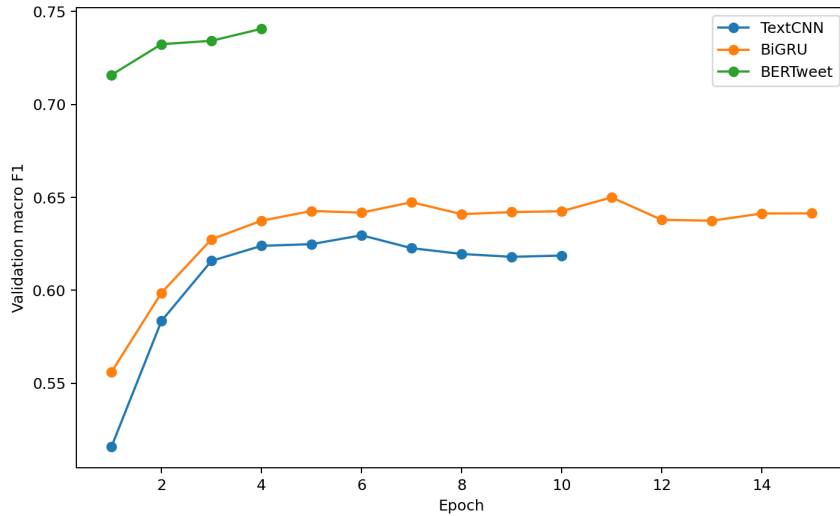


Figure 4: Validation macro-F1 during training for seed 42.

6 Detailed Analysis

6.1 Performance on Different Types of Tweets

The test set was divided into overlapping groups based on observable properties. Table 5 reports accuracy for negations, URLs, mentions, hashtags, emojis, and different text lengths. BERTweet has the best result in every group.

Table 5: Accuracy for selected test subgroups.

Group	Texts	LinearSVC	TextCNN	BiGRU	BERTweet
Negation	2,158	0.572	0.559	0.559	0.696
URL	376	0.636	0.641	0.633	0.697
Mention	4,997	0.632	0.608	0.617	0.732
Hashtag	4,825	0.611	0.608	0.618	0.711
Emoji	809	0.648	0.591	0.585	0.718
1–6 words	828	0.665	0.634	0.643	0.773
7–15 words	5,778	0.620	0.612	0.617	0.721
16–25 words	5,434	0.597	0.584	0.596	0.707
26+ words	244	0.582	0.520	0.553	0.693

The non-transformer systems lose about four to five accuracy points on tweets containing negation, while BERTweet loses about two points compared with its overall accuracy. All models also perform worse as texts become longer. TextCNN has the largest decrease, from 0.634 on very short tweets to 0.520 on texts of 26 words or more. This suggests that selecting only the maximum activation from each convolution may lose information when more content is present.

LinearSVC performs better than the two from-scratch neural systems on tweets containing emojis. The preprocessing converts emojis to textual descriptions, and the word and character TF-IDF features can use parts of these descriptions. However, this explanation should be treated cautiously because emojis were not detected in the training split, so the models had little direct supervised evidence for this group.

6.2 Statistical Tests

The 95% bootstrap confidence interval for BERTweet macro-F1 is [0.710, 0.726]. The intervals for LinearSVC, TextCNN, and BiGRU are [0.600, 0.616], [0.581, 0.598], and [0.591, 0.609], respectively.

McNemar’s test was applied to paired accuracy disagreements. After Bonferroni correction, every comparison involving BERTweet remained significant with an adjusted p-value below 10^{-94} . LinearSVC was also significantly better than TextCNN in accuracy ($p_{adj} = 0.0178$). The difference between LinearSVC and BiGRU was not significant ($p_{adj} = 1.0$), and the difference between TextCNN and BiGRU was also not significant after correction ($p_{adj} = 0.115$).

These tests support two main conclusions. First, BERTweet’s advantage is large enough that it is not explained by normal sampling variation in this test set. Second, the three non-transformer systems are much closer to each other, especially LinearSVC and BiGRU.

6.3 Prediction Agreement and Errors

The four systems correctly classified the same 4,667 tweets and failed together on 1,505 tweets. BERTweet was the only correct system on 1,420 examples. This shows that its improvement is not caused only by small changes in confidence; it solves many tweets that the other approaches cannot classify correctly.

The common errors are likely to include ambiguous wording, missing context, irony, and messages whose label is difficult to infer from the text alone. The project does not attempt to relabel these cases because the official test labels must remain unchanged. Instead, they are kept in the generated error-analysis file so that the behaviour can be inspected later.

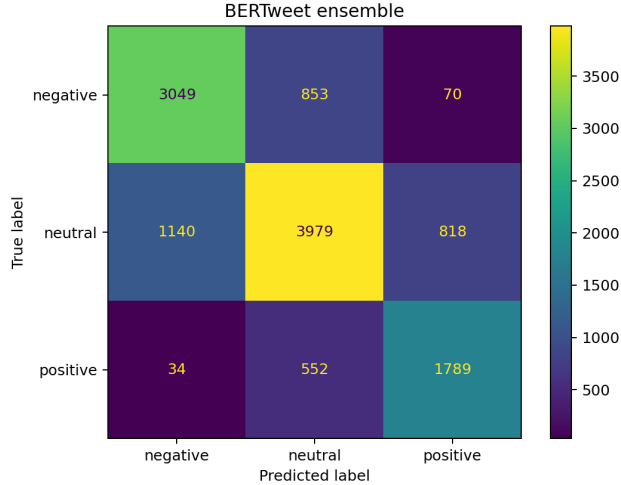


Figure 5: Confusion matrix of the BERTweet ensemble.

The BERTweet confusion matrix shows that direct confusion between positive and negative is uncommon. Most errors pass through the neutral class. For example, a negative tweet is more often predicted as neutral than positive. This is reasonable because neutral is the largest class and often shares vocabulary with the two opinion classes.

7 Efficiency and Practical Trade-offs

BERTweet provides the best predictive quality, but it is also the most expensive system. Table 6 compares training time, inference time for the complete test set, parameter count, and saved model size. Neural values refer to the complete three-model ensemble.

Table 6: Computational cost of the final systems.

Model	Train (s)	Test inference (s)	Parameters	Size (MB)
TF-IDF + LinearSVC	71.6	0.016	0.54M	9.7
TextCNN ensemble	129.8	1.40	8.24M	31.5
BiGRU ensemble	326.8	1.69	13.10M	50.0
BERTweet ensemble	3,202.0	141.27	404.71M	1,544.1

The result creates a clear practical choice. BERTweet should be preferred when prediction quality is the main requirement and GPU resources are available. LinearSVC is more suitable when the model has to process a very large number of texts with limited hardware or low latency. TextCNN and BiGRU require more training than LinearSVC but do not produce a significant accuracy improvement over it in this experiment.

The parameter totals in Table 6 include all three members of each neural ensemble. A single BERTweet model has about 135 million parameters and already reaches approximately 0.713 macro-F1. Therefore, using one transformer instead of the complete ensemble may be a reasonable compromise when storage and inference time are limited.

8 Limitations

The work has several limitations. The dataset contains only English tweets and uses only three sentiment labels. The validation split is small compared with the training set, so checkpoint

selection is based on 2,000 examples. There is also a clear difference in class distribution, average length, and emoji frequency between training and test.

Only a small set of hyperparameters was considered. The purpose was to compare the main model families under a consistent protocol rather than run a large optimization study. TextCNN and BiGRU learn their embeddings from scratch, so the comparison also measures the effect of pretraining, not only architecture. Finally, the ensemble results require three models at inference time. This improves stability and performance but increases storage and computational cost.

9 Reproducibility and Repository

The project was executed from a clean Colab runtime. The notebook downloads the dataset, trains the four systems, evaluates them, generates the tables and figures, and exports the result files. The repository will contain the notebook, dependency list, report, figures, and CSV outputs. The trained BERTweet checkpoints are not included because of their size, but they can be regenerated by executing the notebook from the beginning.

The project repository is available at:

https://github.com/JuanMiguelPinos/Classical_Neural_Models_Sentiment_Analysis

10 Conclusion

This project compared a classical linear model, two neural networks trained from scratch, and a pretrained transformer for multiclass tweet sentiment analysis. BERTweet achieved the best result with a test macro-F1 of 0.718 and was also the most stable model across random seeds. Its advantage was significant against every other system and remained visible for all the linguistic subgroups considered.

The classical LinearSVC model obtained the second-best macro-F1, 0.608, with very low training and inference cost. BiGRU reached a similar result, but the difference between both systems was not statistically significant. TextCNN obtained the lowest final score. These results show that adding a more complex neural architecture does not automatically improve performance when it must learn all text representations from a moderately sized labeled dataset.

The final recommendation depends on the intended use. BERTweet is the best choice when accuracy is the priority and GPU inference is acceptable. LinearSVC is the better option when speed, size, and simplicity are more important. The comparison therefore supports both a high-quality transformer solution and a practical classical alternative.